

For a Gmail OAuth integration, the usual pattern is **not to keep refreshing the access token continuously**. Instead, run a small background worker that:

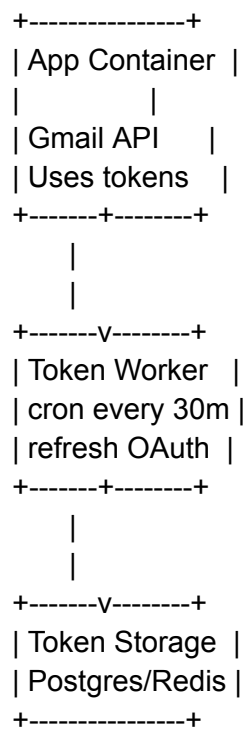
1. Stores the **refresh token** securely.
2. Checks whether an access token is near expiry.
3. Refreshes only when needed.
4. Stores the new access token and expiry time.
5. Uses the token for Gmail API calls.

A cron job inside Docker can do this, but a long-running worker is often cleaner.

Option 1 — Docker + cron refresh worker (simple)

Architecture:

Docker Compose



Example:

token_refresh.sh

```
#!/bin/sh
```

```
curl -X POST http://app:3000/oauth/refresh
```

crontab

```
*/30 * * * * /scripts/token_refresh.sh >> /var/log/oauth-refresh.log 2>&1
```

Dockerfile:

```
FROM alpine:latest
```

```
RUN apk add --no-cache curl busybox-suid
```

```
COPY crontab /etc/crontabs/root
```

```
COPY token_refresh.sh /scripts/token_refresh.sh
```

```
CMD ["crond", "-f", "-l", "2"]
```

This wakes every 30 minutes and asks your app to refresh tokens.

Option 2 — Better: background worker with expiry-aware refresh

Instead of cron guessing:

every 5 minutes:

load token

if expires_at < now + 10 minutes:

refresh()

else:

do nothing

Node.js example:

```
setInterval(async () => {
```

```
  const token = await db.getGoogleToken();
```

```
  const expiresSoon =
```

```
    Date.now() > token.expiresAt - 10 * 60 * 1000;
```

```
  if (expiresSoon) {
```

```
    const newToken = await oauth2Client.refreshAccessToken();
```

```
    await db.saveToken({
```

```
      accessToken: newToken.credentials.access_token,
```

```
      expiresAt: newToken.credentials.expiry_date
```

```
});  
}  
, 5 * 60 * 1000);
```

Benefits:

- no unnecessary refresh calls
- handles restarts
- easier logging
- easier scaling

Option 3 — Local Docker + cloud deployment friendly

Use a persistent volume:

services:

```
gmail-worker:  
  image: your-worker  
  restart: always  
  environment:  
    DATABASE_URL: postgres://...  
    GOOGLE_CLIENT_ID: ...  
    GOOGLE_CLIENT_SECRET: ...  
  volumes:  
    - ./logs:/logs
```

Store:

```
CREATE TABLE oauth_tokens (  
  user_id UUID PRIMARY KEY,  
  access_token TEXT,  
  refresh_token TEXT,  
  expires_at TIMESTAMP  
);
```

Never store only the access token. If the container dies, you lose access.

For a production system I would choose:

Single user / personal automation

- Docker cron worker
- SQLite/Postgres token store

SaaS / multiple Gmail users

- Queue worker (BullMQ, RabbitMQ, SQS)
- Postgres token table
- refresh on demand before API calls

Example:

API request

|

v

Need Gmail?

|

v

Check token expiry

|

+--+---+

| |

valid expired

| |

Gmail refresh

|

v

Gmail

The refresh token is the long-lived credential — treat it like a password. Encrypt it at rest and do not put it in Docker images or environment files committed to source control.